Low Power Edge Inference for Green Computing: An Evaluation of Post-Training Quantization on Apple Silicon

Harshpreet Singh
harshpreet393.be22@chitkara.edu.in
Chitkara University Institute of Engineering and Technology
Chitkara University, Punjab, India

Sameer Kumar
sameer776.be22@chitkara.edu.in
Chitkara University Institute of Engineering and Technology
Chitkara University, Punjab, India

Abstract

Running large language models on edge devices means squeezing them into tight memory budgets, without losing too much accuracy in the process. In this paper, we evaluate post-training quantization (PTQ) for small language models on Apple Silicon using Apple's MLX framework at three precision levels (FP16, INT8, INT4). We benchmark the Qwen3.5 series (0.8B, 2B, 4B parameters) and Phi-3 Mini (3.82B) on an M4 Pro with 24 GB unified memory. INT4 gives us 2.0×–3.8× better throughput over FP16 and cuts memory by 3.5×–3.6×. On the accuracy side, our logit-based full MMLU evaluation (all 57 subjects, 14,042 questions) shows that INT8 is effectively lossless for the 2B and larger models, while INT4 introduces model-dependent drops of 2.39–7.90 percentage points. Even so, the 4B-class models retain strong INT4 accuracy (69.82% for Qwen3.5-4B and 66.97% for Phi-3 Mini). Every model we tested fits within 2.5 GB after INT4 quantization, which opens the door to edge deployment even on devices with only 8 GB of RAM.

1 Introduction

Large language models (LLMs) have reshaped the way we think about natural language processing. Text generation, reasoning, and knowledge retrieval have all taken massive leaps forward Brown et al. [2020], OpenAI [2023]. But this progress comes at a cost. The International Energy Agency projects that data centers will eat up 945 TWh of electricity per year by 2030, roughly 3% of the world's total, with AI workloads being a major driver of that growth International Energy Agency [2025].

1.1 Sustainability Challenge

The environmental footprint of LLMs extends across their entire lifecycle. Training gets most of the attention, but inference is actually the bigger energy consumer in practice, accounting for about 60% of total ML compute in production systems Patterson et al. [2021]. And as models keep getting bigger to push performance forward, energy consumption scales right along with them.

One promising direction is edge computing, which brings inference closer to the user: lower latency, better privacy, and less dependence on data center infrastructure Shi et al. [2016]. The problem is that edge devices are resource-constrained. Most have 8–16 GB of memory, limited compute, and tight power budgets. Just loading a 4-billion-parameter model in FP16 takes around 8 GB of memory for weights alone, which is more than a lot of edge devices can spare.

1.2 Quantization as a Solution

This is where post-training quantization (PTQ) comes in. PTQ reduces weight precision from 16-bit or 32-bit floating-point down to 8-bit or even 4-bit integers Han et al. [2016]. The key advantage over quantization-aware

training (QAT) is that PTQ works directly on pre-trained weights, with no retraining needed. That makes it a natural fit for getting existing models onto edge devices.

That said, several practical questions remain unanswered:

- How do different quantization bit-widths affect throughput, memory, and accuracy on Apple Silicon?
- How does model size influence quantization sensitivity?

1.3 Contributions

We set out to answer these questions through hands-on experiments on an Apple M4 Pro. Here is what we contribute:

1. **Quantization analysis**: We benchmark MLX at three precision levels (FP16, INT8, INT4) across the Qwen3.5 series and Phi-3 Mini, measuring how each level affects throughput, memory usage, and accuracy.
2. **Accuracy evaluation**: We use logit-based full-MMLU scoring on all 57 subjects (14,042 questions), which reduces the 95% confidence interval around overall accuracy to about ± 0.8 pp and lets us measure quantization effects with much higher confidence.
3. **Energy estimation**: We report CodeCarbon-based energy estimates for every configuration, while being upfront about the limitations of software-based power estimation on Apple Silicon.

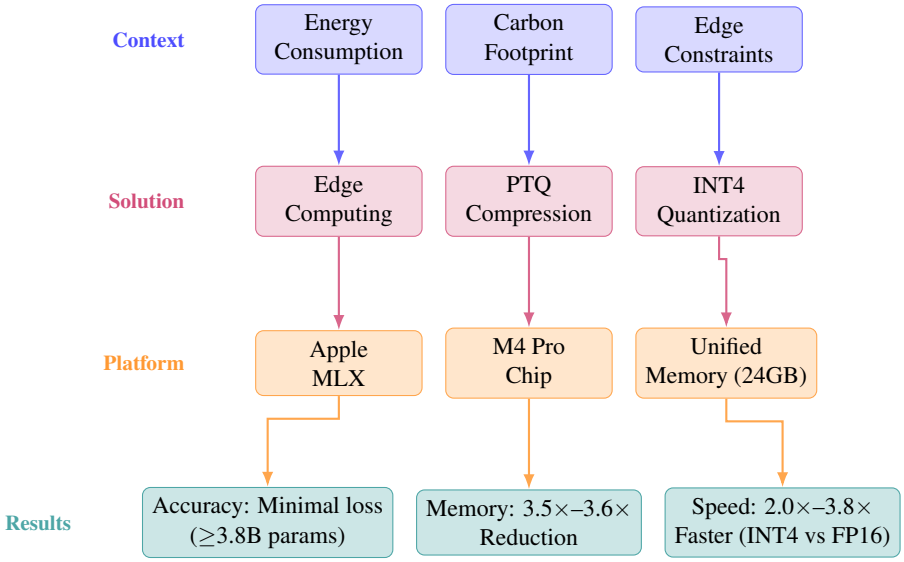

Figure 1: Conceptual framework for PTQ-based edge AI deployment on Apple Silicon.

Figure 1 shows the big picture of what we are doing: taking the growing energy problem of LLM inference, applying PTQ compression on Apple Silicon, and measuring what we actually get in terms of throughput, memory, and accuracy.

2 Background

The environmental cost of deploying LLMs is no longer a footnote; it is a first-order research problem. Training gets the headlines, but inference is where most of the energy actually goes: roughly 60% of all ML compute in production systems Patterson et al. [2021]. Running inference at the edge can help (less reliance on data centers, lower latency, better privacy Shi et al. [2016]), but edge devices come with hard limits: usually 8–16 GB of memory and far less compute than a cloud GPU.

2.1 Model Compression Techniques

Model compression is the umbrella term for techniques that shrink neural networks while keeping accuracy mostly intact Cheng et al. [2017]. The main approaches are:

2.1.1 Quantization

Quantization maps continuous floating-point values to discrete low-bit representations. For a weight tensor $\mathbf{W} \in \mathbb{R}^{n \times m}$, quantization to b bits involves:

$$\mathbf{W}_q = \text{clamp}\left(\text{round}\left(\frac{\mathbf{W}}{s}\right), -2^{b-1}, 2^{b-1} - 1\right) \tag{1}$$

where s is the scale factor. The dequantized approximation is:

$$\hat{\mathbf{W}} = s \cdot \mathbf{W}_q \tag{2}$$

The quantization error $\|\mathbf{W} - \hat{\mathbf{W}}\|$ directly affects model accuracy.

2.1.2 Pruning

Pruning removes redundant parameters based on importance criteria by setting weights to zero. For weight matrix $\mathbf{W}$, pruning is expressed as:

$$\mathbf{W}_{\text{pruned}} = \mathbf{W} \odot \mathbf{M}, \quad \mathbf{M}_{ij} = \begin{cases} 1 & \text{if } |W_{ij}| > \tau \\ 0 & \text{otherwise} \end{cases} \tag{3}$$

where τ is a threshold and $\odot$ denotes element-wise multiplication.

2.1.3 Knowledge Distillation

Knowledge distillation trains a smaller "student" model to mimic a larger "teacher" model Hinton et al. [2015]. It works well, but you need access to the teacher during training, and in many real-world scenarios, that access simply is not available, which makes distillation less practical for post-deployment compression.

2.2 Post-Training Quantization for Edge Deployment

PTQ has several things going for it on the edge: (1) you do not need to retrain anything; (2) the quantization process takes minutes, not days; (3) modern edge chips have built-in support for low-precision arithmetic; and (4) going from FP16 to 4-bit cuts memory usage by roughly $4\times$. All of this makes PTQ the most practical path for getting pre-trained models onto constrained hardware.

3 Related Work

3.1 Post-Training Quantization Methods

PTQ methods differ mainly in how cleverly they handle quantization error. Here are the key ones:

3.1.1 INT8 Uniform Quantization

The simplest approach: map floating-point values to evenly spaced integer levels. LLM.int8() Dettmers et al. [2022] improved on naive INT8 by introducing a mixed-precision decomposition that identifies the outlier feature dimensions that need higher precision and keeps those in FP16:

$$\mathbf{Y} = \mathbf{X}_{\text{normal}}\mathbf{W}_{\text{int8}} + \mathbf{X}_{\text{outlier}}\mathbf{W}_{\text{fp16}} \tag{4}$$

where $\mathbf{X}_{\text{outlier}}$ contains dimensions whose values exceed a threshold. The result is roughly $2\times$ memory savings with minimal accuracy loss.

SmoothQuant Xiao et al. [2023] tackles the problem from a different angle. Instead of handling outliers at inference time, it shifts the quantization difficulty from activations to weights using a per-channel scaling trick. The math works out to be equivalent, but the quantized model is easier to run efficiently, especially for large models where activation outliers are common.

3.1.2 GPTQ: Optimal Brain Quantization

GPTQ Frantar et al. [2023] brings the Optimal Brain Surgeon (OBS) idea LeCun et al. [1990] into the LLM world. The core insight is that when you quantize one weight, you can adjust the remaining weights to compensate for the error you just introduced:

$$\delta_{\mathbf{W}} = -\frac{w_q - w}{[\mathbf{H}^{-1}]_{qq}} \cdot \mathbf{H}^{-1}_{:,q} \tag{5}$$

where $\mathbf{H}$ is the Hessian of the layer's output, w is the original weight, w_q is quantized, and q indexes the position being quantized. The catch is computing $\mathbf{H}^{-1}$, which is $O(d^3)$ for d inputs. GPTQ gets around this with Cholesky decomposition and lazy batch updates, but calibration is still expensive.

3.1.3 AWQ: Activation-Aware Weight Quantization

AWQ Lin et al. [2024] takes a different philosophy: instead of fixing errors after quantization, it protects the weights that matter most *before* quantizing. It identifies important weights by looking at activation magnitudes and optimizes a modified loss:

$$s^* = \arg\min_s \mathscr{L}(\mathbf{W}, s), \quad \mathscr{L} = \sum_{i,j} \left| \mathbf{X}_{i,j} \cdot (w_{i,j} - \hat{w}_{i,j}) \right| \tag{6}$$

where s is a per-channel scaling factor, $\mathbf{X}$ represents activations from calibration data, and the scaling is applied as:

$$\hat{\mathbf{W}} = \text{quant}(\mathbf{W} \cdot \text{diag}(s)) \cdot \text{diag}(s)^{-1} \tag{7}$$

By concentrating quantization precision on the most activation-sensitive weights, AWQ keeps accuracy degradation remarkably low. It received the Best Paper Award at MLSys 2024.

3.1.4 INT4 Weight-Only Quantization

The simplest 4-bit approach: just quantize all weights to INT4 uniformly, with no error compensation at all. It is fast and easy, but the accuracy hit can be significant, especially for smaller models where every parameter carries more weight (pun intended). For models under 4B parameters, you really need to check whether the accuracy loss is acceptable for your use case.

3.1.5 Block Quantization (Q4_K_M)

The GGUF format from llama.cpp Gerganov [2023] takes a more nuanced approach to 4-bit quantization. Schemes like Q4_K_M split weight matrices into blocks (typically 32 or 256 elements) and store per-block scale factors and minimums. Some blocks get 4-bit precision, others get 6-bit, depending on their importance.

The goal is to preserve more accuracy than naive INT4, but the trade-off is extra dequantization overhead at inference time, since each block has to be rescaled independently.

3.2 Quantization for Edge Deployment

A growing body of work has looked at deploying quantized LLMs on edge hardware:

llama.cpp Gerganov [2023] introduced the GGUF format with various quantization schemes (Q4_0, Q4_K_M, Q5_K_M, etc.) optimized for CPU inference.

MLC-LLM MLC Team [2023] provides a compilation-based approach for deploying quantized LLMs across diverse hardware platforms, including mobile devices and Apple Silicon.

Huang et al. Huang and Wang [2025] evaluated LLM inference on edge devices, measuring energy efficiency across 28 quantized models on Raspberry Pi 4.

Benazir and Lin Benazir and Lin [2025] profiled LLM inference on Apple Silicon from a quantization perspective, identifying optimal configurations for M-series chips.

Kurtic et al. Kurtic et al. [2024] conducted a systematic evaluation of accuracy–performance trade-offs across multiple quantization formats, finding that the choice of quantization scheme matters more than the bit-width alone.

Jin et al. Jin et al. [2024] provided a comprehensive evaluation of quantization strategies for LLMs, comparing weight-only, weight-activation, and mixed-precision approaches.

Where our work fits in: we systematically evaluate MLX quantization (FP16, INT8, INT4) on Apple Silicon using some of the newest small models (Qwen3.5 series and Phi-3 Mini), and we include energy estimation and logit-based accuracy evaluation alongside the usual throughput and memory metrics.

3.3 Small Language Models

On the edge, you need models that are small enough to actually fit. The two families we focus on are:

Phi Series Javaheripi and Bubeck [2023], Abdin et al. [2024]: Microsoft showed with the Phi family that if you are really careful about training data, a small model can punch well above its weight class. Phi-3 Mini packs 3.82B parameters and puts up benchmark numbers you would not expect from a model that small.

Qwen Series Yang et al. [2025]: Alibaba's Qwen3.5 series builds on the Qwen3 architecture, ranging from tiny (0.8B) to massive (397B parameters). The smaller variants we use (0.8B, 2B, and 4B) feature GQA attention with RoPE embeddings, SwiGLU activation, and a dual-mode inference design (thinking and non-thinking modes). They deliver surprisingly competitive accuracy for their size, which is exactly what you want for edge deployment.

3.4 Energy Measurement in ML

Measuring how much energy ML workloads consume is harder than it sounds:

CodeCarbon ML CO2 Impact [2023]: A Python package that estimates emissions by monitoring CPU, GPU, and RAM power draw. Easy to integrate, supports regional carbon intensity data, and works across platforms, but it relies on power models, not direct measurements.

RAPL David et al. [2010]: Intel's built-in interface for real-time power measurement at the processor level. Widely used in ML energy research, but it is Intel/AMD only, with no Apple Silicon support.

Hardware Power Meters Conti et al. [2023]: The gold standard for accuracy, but you need physical instrumentation plugged into the wall.

This creates a real problem for our setup. On Apple Silicon, RAPL is not available, and the unified memory architecture makes it hard to attribute power to individual components. CodeCarbon on macOS falls back to TDP-based models, essentially educated guesses based on the chip's rated power. We discuss what this means for our energy results in Section 4.4.4.

Table 1: Comparison of Related Work in PTQ, Edge Deployment, and Energy Measurement

Category	Method/Study	Year	Key Features	Limitations	Hardware	Ref
PTQ Methods						
	LLM.int8()	2022	Mixed-precision, outlier detection	Computational overhead	Hardware-agnostic	Dettmers et al. [2022]
	SmoothQuant	2023	Activation-weight balancing	Requires calibration	Hardware-agnostic	Xiao et al. [2023]
	GPTQ	2023	OBS-based, Hessian compensation	Expensive calibration	Hardware-agnostic	Frantar et al. [2023]
	AWQ	2024	Activation-aware, salient weights	Requires calibration data	Hardware-agnostic	Lin et al. [2024]
Edge Frameworks						
	llama.cpp	2023	GGUF format, CPU-optimized	Limited GPU initially	CPU, GPU, Metal	Gerganov [2023]
	MLC-LLM	2023	TVM compilation, cross-platform	Complex setup	GPU, Metal, Vulkan	MLC Team [2023]
	Huang et al.	2025	28 models, Raspberry Pi 4	Hardware-specific	Edge devices	Huang and Wang [2025]
	Benazir & Lin	2025	Apple Silicon profiling	Apple-only	Apple Silicon	Benazir and Lin [2025]
	Kurtic et al.	2024	Accuracy-performance trade-offs	Focused on BF16/FP16	Multi-platform	Kurtic et al. [2024]
	Jin et al.	2024	Comprehensive quantization eval	No energy measurement	Multi-platform	Jin et al. [2024]
Small Language Models						
	Phi Series	2023–2024	Compact, high capability	Limited general knowledge	Universal	Javaheripi and Bubeck [2023], Abdin et al. [2024]
	Qwen Series	2025–2026	0.8B–397B range, multilingual	Larger models need resources	Universal	Yang et al. [2025]
Energy Measurement						
	CodeCarbon	2023	Multi-platform, easy integration	±10–15% accuracy	Multi-platform	ML CO2 Impact [2023]
	RAPL	2010	Intel MSR-based, real-time	Intel/AMD only	Intel/AMD CPUs	David et al. [2010]
	Hardware Meters	2023	Physical measurement, accurate	External hardware needed	External device	Conti et al. [2023]

4 Methodology

4.1 Experimental Design

Our setup is fairly straightforward, as illustrated in Figure 2:

1. **Model selection**: We evaluate the Qwen3.5 series (0.8B, 2B, 4B) and Phi-3 Mini, representing diverse parameter counts and architectures.
2. **Quantization sweep**: We benchmark MLX at three precision levels (FP16, INT8, INT4) for each model.
3. **Comprehensive metrics**: We measure throughput, memory footprint, accuracy, and estimated energy consumption.

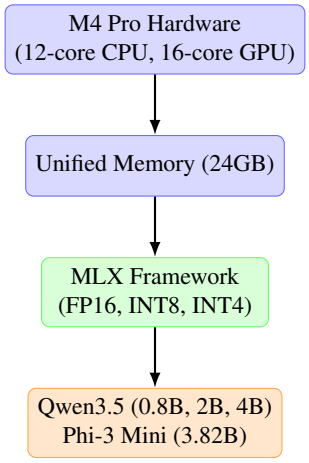

Figure 2: System architecture for PTQ evaluation on Apple Silicon using MLX.

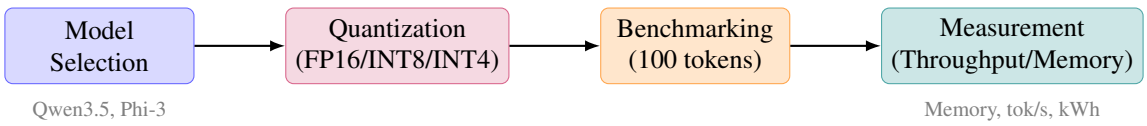

Figure 3: Experimental workflow for generation benchmarking on Apple Silicon. Accuracy is measured separately with a full logit-based MMLU pass.

Figure 3 lays out the throughput, memory, and energy pipeline: pick a model, quantize it, benchmark generation, and collect the system metrics. Accuracy is evaluated separately using the full MMLU sweep described

below.

4.2 Models

We picked these models for three reasons. First, **size diversity**: the Qwen3.5 series goes from 873M to 4.66B parameters, so we can see how model scale affects quantization trade-offs. Second, **they are genuinely good**: both families post strong benchmark numbers while staying small enough for edge hardware. Third, **architectural diversity**: Qwen3.5 uses GQA attention with RoPE embeddings, while Phi-3 is a straightforward dense transformer. This lets us test whether quantization sensitivity depends on model design.

4.2.1 Qwen3.5 Series

The Qwen3.5 series Yang et al. [2025] from Alibaba are efficient language models with hybrid reasoning capabilities:

- **Qwen3.5-0.8B**: 873M parameters, optimized for extreme efficiency
- **Qwen3.5-2B**: 2.27B parameters, balanced size–performance trade-off
- **Qwen3.5-4B**: 4.66B parameters, higher accuracy with greater resource requirements

Key features include GQA attention, RoPE embeddings, SwiGLU activation, and dual-mode reasoning (thinking/non-thinking). All Qwen3.5 experiments were conducted using MLX.

4.2.2 Phi-3 Mini

Phi-3 Mini Abdin et al. [2024] is a 3.82-billion-parameter dense transformer from Microsoft: 3072 hidden dimensions, 32 attention heads, 32 layers, 128K context length. Experiments were conducted using MLX.

4.3 Hardware Platform

Everything runs on Apple Silicon. The M4 Pro's killer feature for LLM inference is its unified memory, where CPU and GPU share the same physical RAM, so there is no data transfer bottleneck. For models that are heavily memory-bandwidth-bound (which LLMs almost always are during generation), this makes a real difference. Full specs are in Section 4.5.1.

4.4 Evaluation Metrics

We measure four things for each model–quantization pair. Throughput and energy numbers are the mean of 10 runs after one warmup run. Accuracy is computed deterministically from a single full-MMLU sweep over the same 14,042 questions for every configuration. Standard deviations stayed below 3% for throughput across all configurations, which tells us the M4 Pro was thermally stable during our tests. We omit error bars for readability but note this variability bound.

4.4.1 Throughput

We measure inference throughput during generation of 100 tokens with a 256-token input context:

$$\text{Throughput} = \frac{\text{Number of Tokens Generated}}{\text{Total Generation Time}} \tag{8}$$

We report the mean over 10 runs after one warmup run to ensure model weights are loaded into the unified memory cache.

4.4.2 Memory Footprint

We measure peak memory consumption during inference, which includes model weights (quantized or full-precision), KV cache, activation buffers, and runtime overhead.

4.4.3 Task Accuracy

For accuracy, we use the full MMLU test split Hendrycks et al. [2021]: all 57 subjects and 14,042 questions, grouped into STEM, Humanities, Social Sciences, and Other. One important design choice: instead of generating text and trying to parse the answer (which is a nightmare with models that have thinking modes), we use **logit-based scoring**. For each multiple-choice question, we just look at the model's next-token log-probabilities for A, B, C, and D, and pick the highest. It is deterministic, you never get an unparseable response, and it matches the multiple-choice evaluation style used in lm-evaluation-harness Gao et al. [2024].

At this sample size, the binomial 95% confidence interval around overall accuracy is roughly ± 0.8 percentage points, which is tight enough to distinguish the multi-point INT4 drops we observe from ordinary sampling noise.

4.4.4 Energy Estimation

We estimate energy consumption using CodeCarbon ML CO2 Impact [2023]:

$$E_{\text{inference}} = \int_{t_0}^{t_1} P(t)\, dt \tag{9}$$

where $P(t)$ is instantaneous power consumption.

Here is the catch: CodeCarbon on Apple Silicon cannot read actual hardware power sensors (RAPL is Intel-only). It falls back to a TDP-based model that assumes roughly constant power draw (about 42.5 W for the CPU and 6 W for RAM). Because the estimated power is essentially a constant, **the energy numbers end up being roughly proportional to wall-clock time**. In other words, when we report "energy savings," we are mostly measuring the same thing as throughput gains from a different angle. We include these numbers because they are still useful for rough comparisons, but they should not be taken as precise energy measurements. Getting real power data on Apple Silicon would require either an external power meter or running Apple's `powermetrics` with root access.

4.5 Experimental Setup

4.5.1 Hardware

All experiments were conducted on:

- **Platform**: Apple MacBook Pro (M4 Pro)
- **Processor**: Apple M4 Pro (12-core CPU, 16-core GPU)
- **Memory**: 24 GB unified memory (LPDDR5X)
- **Storage**: 512 GB SSD
- **OS**: macOS Sequoia 15.3

The unified memory design means the CPU and GPU share the same physical memory pool, so there is no copying data back and forth, which is a significant advantage for memory-hungry workloads like LLM inference.

4.5.2 Software

- **Framework**: MLX 0.31.0 (Apple-native)
- **Quantization**: MLX built-in quantization (FP16, INT8, INT4)

- **Energy estimation**: CodeCarbon 3.2.3
- **Accuracy evaluation**: Custom logit-based MMLU scorer using Hugging Face Transformers

4.5.3 Benchmarking Protocol

- Generation length: 100 tokens
- Input context: 256 tokens
- Temperature: 0.0 (greedy decoding)
- Warmup: 1 run (discarded) before 10 measured runs
- Accuracy benchmark: full MMLU test split (57 subjects, 14,042 questions) scored by next-token logits over A/B/C/D
- Random seed: 42 for reproducibility
- Thinking mode: disabled for Qwen3.5 models during generation and MMLU scoring

5 Experiments and Results

Here are the results from running all four models through MLX at three precision levels (FP16, INT8, INT4) on our M4 Pro. Throughput and energy numbers are averages of 10 runs after a warmup pass; accuracy numbers come from a single deterministic sweep over the full 14,042-question MMLU test set. All raw data and benchmark scripts are in the project repository for anyone who wants to reproduce or dig deeper.

5.1 Memory Footprint

Table 2 shows memory requirements across all configurations.

Table 2: Memory footprint (GB) across MLX quantization levels.

Model	FP16	INT8	INT4
Qwen3.5-0.8B	1.40	0.74	0.39
Qwen3.5-2B	3.51	1.86	0.99
Qwen3.5-4B	7.83	4.16	2.20
Phi-3 Mini	7.12	3.78	2.00

INT4 delivers 3.5×–3.6× memory reduction over FP16 across the board. Every model fits within 2.5 GB at INT4, making edge deployment feasible even on devices with only 8 GB of RAM.

5.2 Quantization Method Comparison

Table 3 compares quantization configurations for Qwen3.5-0.8B, illustrating the trade-offs between precision, throughput, and estimated energy.

Table 3: Quantization comparison for Qwen3.5-0.8B on M4 Pro (100 tokens generated).

Method	Size (GB)	tok/s	ms/tok	Speedup
FP16	1.40	104.69	9.55	1.0×
INT8	0.74	166.08	6.02	1.6×
INT4	0.39	213.11	4.69	2.0×

The pattern is clean: lower bit-width means higher throughput. INT4 runs 2.0× faster than FP16 for this model, with INT8 sitting in between at 1.6×. Each step down in precision roughly halves memory while boosting

throughput, a trade-off that is well worth it if accuracy holds up (which we examine in Section 5.5).

5.3 Throughput Analysis

Table 4 presents throughput and memory across all MLX configurations.

Table 4: MLX throughput and memory across quantization levels (100 tokens generated).

Model	FP16		INT8		INT4		Speedup
	tok/s	GB	tok/s	GB	tok/s	GB	(INT4/FP16)
Qwen3.5-0.8B	104.69	1.40	166.08	0.74	213.11	0.39	2.0×
Qwen3.5-2B	52.08	3.51	87.42	1.86	131.86	0.99	2.5×
Qwen3.5-4B	24.15	7.83	42.49	4.16	71.21	2.20	2.9×
Phi-3 Mini	23.23	7.12	45.64	3.78	88.40	2.00	3.8×

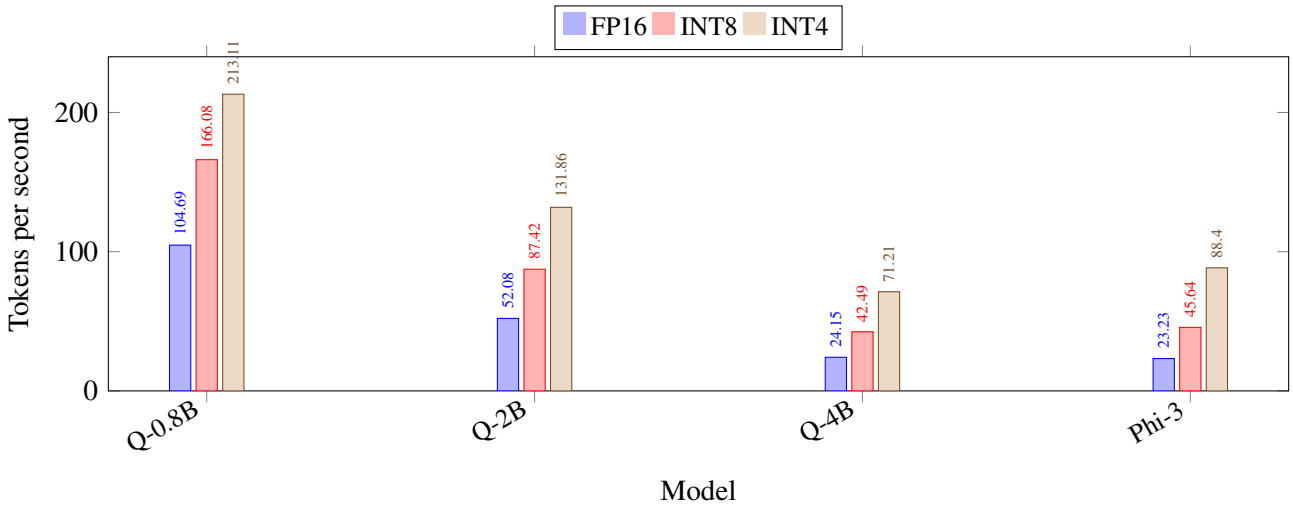

Figure 4: MLX throughput across quantization levels. Larger models exhibit greater relative speedup from INT4 quantization, likely because they are more memory-bandwidth-bound at FP16.

Two things jump out from Table 4 and Figure 4. First, the bigger the model, the more you gain from INT4: the speedup goes from 2.0× for the 0.8B model all the way up to 3.8× for Phi-3 Mini. This makes sense because larger models are more memory-bandwidth-bound at FP16, so cutting bandwidth by 4× helps them more. Second, there is an interesting reversal: Phi-3 Mini (3.82B) actually runs faster at INT4 (88.40 tok/s) than the larger Qwen3.5-4B (71.21 tok/s, 4.66B). Phi-3's simpler dense architecture seems to be more compute-efficient per parameter.

5.4 Energy Estimation

We want to reiterate the caveat from Section 4.4.4: on Apple Silicon, CodeCarbon's energy numbers are derived from TDP-based power models, so they track inference time more than actual power draw. We include them because they are the best we can do without hardware instrumentation, but keep this in mind as you read the tables.

Table 5 shows the estimated energy across MLX quantization levels.

5.5 Accuracy Retention Under Quantization

Table 6 presents full MMLU accuracy across all models and quantization levels, measured using logit-based scoring on all 57 subjects (14,042 questions).

Table 5: MLX estimated energy across quantization levels (100 tokens). Energy values are CodeCarbon estimates based on TDP models, not direct measurements.

Model	FP16		INT8		INT4		Est. Energy
	tok/s	kWh	tok/s	kWh	tok/s	kWh	Ratio
Qwen3.5-0.8B	104.69	0.000013	166.08	0.000008	213.11	0.000006	2.2×
Qwen3.5-2B	52.08	0.000026	87.42	0.000015	131.86	0.000010	2.6×
Qwen3.5-4B	24.15	0.000056	42.49	0.000032	71.21	0.000019	2.9×
Phi-3 Mini	23.23	0.000058	45.64	0.000030	88.40	0.000015	3.9×

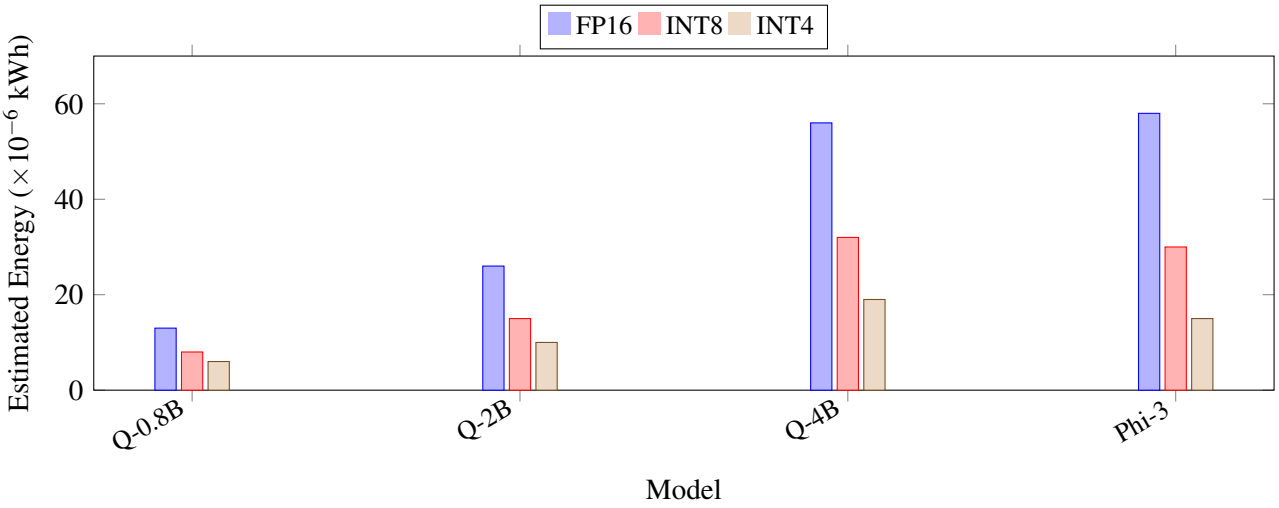

Figure 5: Estimated energy consumption across MLX quantization levels. Because CodeCarbon uses constant TDP-based power estimates on Apple Silicon, these values are approximately proportional to inference time (see Section 4.4.4).

Table 6: Full MMLU accuracy (%) across quantization levels (logit-based scoring, all 57 subjects, $n = 14{,}042$). Binomial 95% confidence intervals around overall accuracy are approximately ± 0.8 pp.

Model	Quant	Overall	STEM	Human.	Social	Other
Qwen3.5-0.8B	FP16	46.31	44.50	39.06	52.63	52.88
Qwen3.5-0.8B	INT8	45.27	42.63	38.66	51.12	52.21
Qwen3.5-0.8B	INT4	43.73	40.50	38.13	50.02	49.22
Qwen3.5-2B	FP16	56.59	53.95	49.39	64.40	62.35
Qwen3.5-2B	INT8	56.54	53.92	49.03	64.34	62.82
Qwen3.5-2B	INT4	48.68	45.67	42.44	54.99	54.94
Qwen3.5-4B	FP16	73.11	70.38	66.16	81.11	78.38
Qwen3.5-4B	INT8	73.07	70.31	66.18	80.96	78.42
Qwen3.5-4B	INT4	69.82	66.48	61.64	79.10	76.32
Phi-3 Mini	FP16	69.36	61.72	64.00	79.07	75.52
Phi-3 Mini	INT8	69.29	61.91	63.93	78.88	75.29
Phi-3 Mini	INT4	66.97	59.44	62.02	76.49	72.56

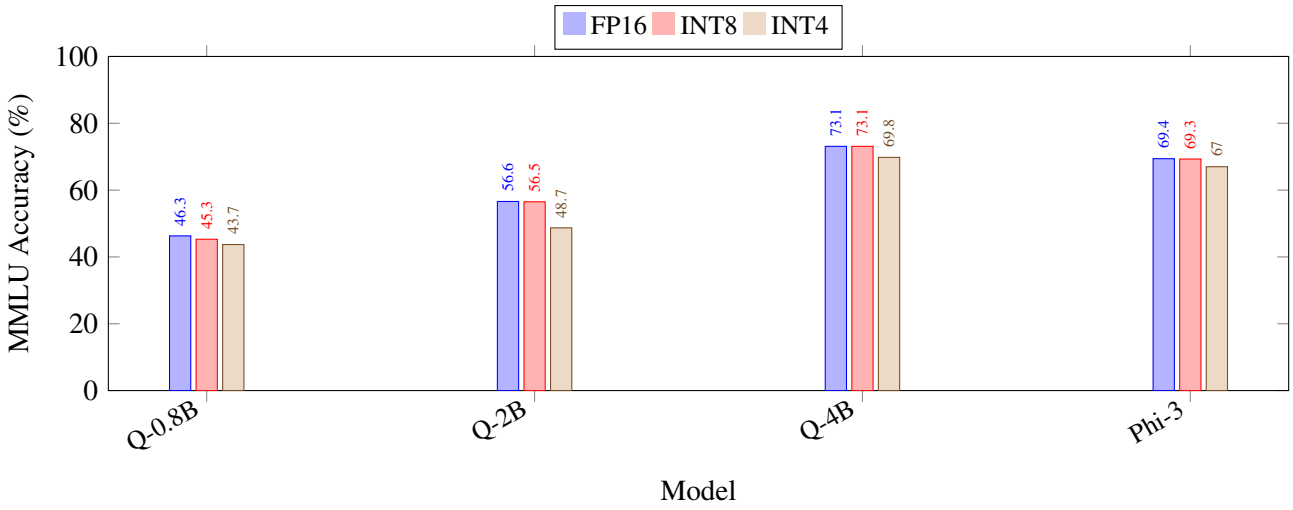

Figure 6: MMLU accuracy across quantization levels on the full 57-subject test set ($n = 14{,}042$). The larger sample makes the INT4 degradation clear, while INT8 remains nearly lossless for the 2B and larger models.

The full benchmark changes the statistical picture. With $n = 14{,}042$, overall accuracy has a 95% confidence interval of roughly ± 0.8 pp, so the INT4 drops are no longer explainable as small-sample noise. Here is what the data shows:

1. **INT8 is essentially lossless for the 2B and larger models**: relative to FP16, the drop is just 0.04 pp for Qwen3.5-2B, 0.04 pp for Qwen3.5-4B, and 0.07 pp for Phi-3 Mini. Qwen3.5-0.8B is the only model with a clearly visible INT8 shift (1.04 pp).
2. **INT4 introduces real but model-dependent degradation**: the full-MMLU drop is 2.59 pp for Qwen3.5-0.8B, 7.90 pp for Qwen3.5-2B, 3.29 pp for Qwen3.5-4B, and 2.39 pp for Phi-3 Mini. The 4B-class models remain usable at INT4, but the loss is no longer negligible.
3. **Category structure is stable under quantization**: Social Sciences is the strongest category for every configuration. The largest INT4 degradation appears in Qwen3.5-2B, where all category drops fall between 6.95 and 9.41 pp. Qwen3.5-4B and Phi-3 Mini still post strong INT4 category scores, especially in Social Sciences and Other.
4. **Accuracy still scales with model quality**: within the Qwen3.5 family, FP16 accuracy rises from 46.31% to 56.59% to 73.11%. INT4 follows the same pattern, rising from 43.73% to 48.68% to 69.82%. The benefits of a stronger base model largely survive quantization.

5.6 Energy–Accuracy Trade-off

To compare efficiency and quality directly, we define *accuracy depreciation* as the full-MMLU drop relative to the FP16 model of the same family. We define *estimated energy reduction* as $100 \times (1 - E_q/E_{\text{FP16}})$ for the same pair. Figure 7 plots these two deltas together. Because CodeCarbon's estimate on Apple Silicon is largely a proxy for runtime, the vertical axis should be read as relative efficiency gain rather than hardware-measured power savings.

Figure 7 makes the practical trade-off easier to read than the raw tables. INT8 for Qwen3.5-2B, Qwen3.5-4B, and Phi-3 Mini sits in the low-loss, high-gain corner. These three settings cut estimated energy by 40.4–49.1% while losing at most 0.07 pp on full MMLU. Among the aggressive INT4 configurations, Phi-3 Mini gives the strongest result: 73.7% lower estimated energy for a 2.39 pp accuracy drop. Qwen3.5-4B INT4 is close behind at 66.1% lower energy for a 3.29 pp drop. The clear outlier is Qwen3.5-2B INT4, which still saves 60.5% energy but pays for it with the largest accuracy depreciation in the study (7.90 pp).

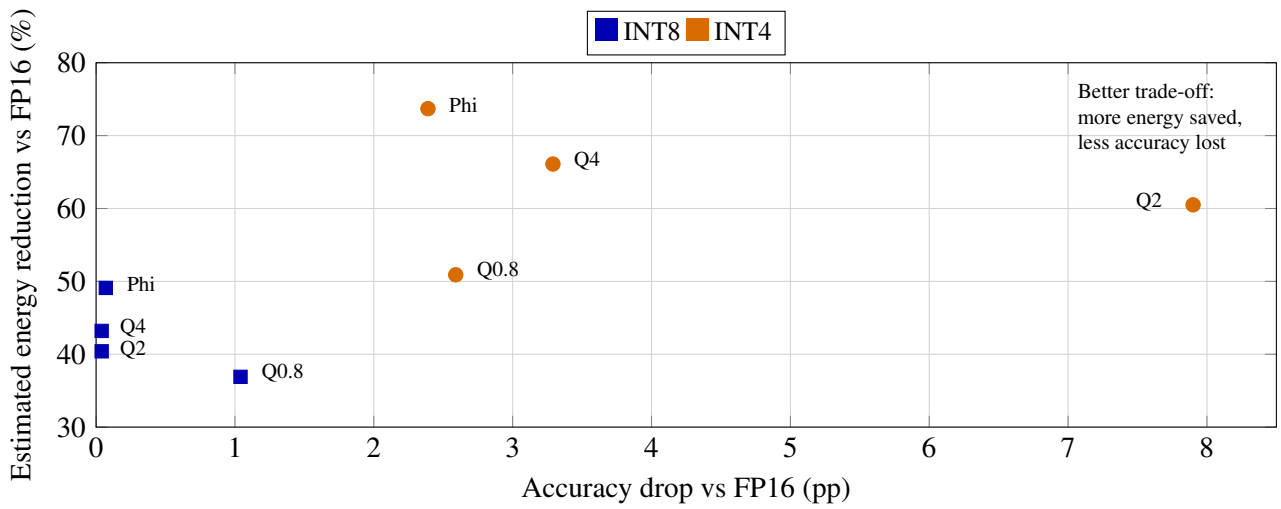

Figure 7: Energy–accuracy trade-off relative to each model's FP16 baseline. Upper-left points are preferable: less full-MMLU accuracy loss for more estimated energy reduction. INT8 occupies the low-loss region, while Phi-3 Mini INT4 and Qwen3.5-4B INT4 offer the strongest aggressive-quantization trade-offs.

5.7 Experimental Limitations

1. **Energy estimation**: As discussed in Section 4.4.4, CodeCarbon on Apple Silicon uses TDP-based power models. Energy values are approximate and correlate strongly with inference time.
2. **Benchmark coverage**: Full MMLU removes the small-sample problem, but it is still a single multiple-choice benchmark. It does not measure open-ended generation quality, coding, tool use, safety, or long-context reasoning.
3. **Single device**: All results are from a single M4 Pro unit; chip-to-chip variation and thermal throttling under sustained load are not characterized.
4. **Short generation length**: Benchmarks use 100-token generation; throughput and energy characteristics may differ for longer sequences, streaming workloads, or batched inference.

5.8 Recommendations for Practitioners

Based on our experimental findings:

1. **Quantization level**: INT4 provides the best throughput–memory trade-off for 4B-class models when a 2–3 pp MMLU drop is acceptable (Figure 7). For smaller models or accuracy-sensitive deployments, INT8 is the safer default.
2. **Model selection**: Qwen3.5-4B offers the best absolute accuracy at every precision, while Phi-3 Mini INT4 delivers the strongest aggressive energy–accuracy trade-off: 73.7% estimated energy reduction for only a 2.39 pp MMLU drop.

6 Discussion

6.1 Why Unified Memory Matters More Than You Think

One of the clearer takeaways from our experiments is that Apple Silicon's unified memory architecture is not just a marketing feature. It genuinely changes the quantization story. When CPU and GPU share the same memory pool, reducing weight precision translates directly into less memory bandwidth consumed. And since memory bandwidth is the bottleneck for autoregressive generation (you are reading one token's worth of weights per step), this matters a lot.

This is why we see the INT4 speedup ratio increase with model size (Table 4): bigger models are more bandwidth-starved at FP16, so they benefit more from the 4× bandwidth reduction that INT4 gives you. Benazir and Lin Benazir and Lin [2025] saw the same pattern independently and found that Apple Silicon amplifies quantization benefits compared to discrete-GPU systems, where PCIe transfer overhead eats into some of the bandwidth savings.

6.2 The Accuracy Question

Our full MMLU results sharpen the answer. INT8 is basically free for the 2B and larger models on this setup: the change from FP16 is only 0.04 pp for Qwen3.5-2B and Qwen3.5-4B, and 0.07 pp for Phi-3 Mini. INT4 is a different story. The accuracy loss is real for every model, but its magnitude depends strongly on the model: 2.59 pp for Qwen3.5-0.8B, 7.90 pp for Qwen3.5-2B, 3.29 pp for Qwen3.5-4B, and 2.39 pp for Phi-3 Mini.

That result is important because it means there is no magic parameter threshold where INT4 suddenly becomes free. Scale helps, especially within the Qwen3.5 family, but architecture and training recipe matter too. This still broadly lines up with what Kurtic et al. Kurtic et al. [2024] and Jin et al. Jin et al. [2024] reported: quantization sensitivity is model-dependent rather than determined by bit-width alone. The practical rule is narrower and more defensible: on Apple Silicon, INT4 is a strong default when throughput and memory matter most and a modest 2–3 pp drop is acceptable for 4B-class models; INT8 is the safer default when the accuracy budget is tight.

6.3 The Energy Measurement Problem

We have been upfront about the limitations of our energy data, but it is worth discussing what we *can* reasonably infer. CodeCarbon's TDP model assumes roughly constant power draw, which is clearly an approximation. But here is the thing: during steady-state token generation, the M4 Pro is probably pulling a fairly consistent amount of power regardless of whether the weights are INT4 or FP16, since the chip is still saturating its memory bus either way. If that assumption holds, then faster inference really does mean less energy per token, and our estimates are directionally correct even if the exact numbers are off.

The interesting open question is whether INT4 workloads actually use *less* power than FP16 at the hardware level. INT4 needs fewer compute operations but similar memory bandwidth. We do not have the data to answer this, and we flag it as one of the most valuable directions for future work.

6.4 Limitations

6.4.1 Hardware Specificity

Everything we measured is specific to one chip, the M4 Pro:

- NVIDIA GPUs with separate VRAM may show different quantization trade-offs due to PCIe bottlenecks
- Mobile SoCs (e.g., Apple A-series) have less memory and different thermal profiles
- Results may not generalize to other Apple Silicon variants (M1, M2, M3, or base M4)

6.4.2 Model Coverage

We only tested four models from two families, so there are gaps:

- Qwen3.5 thinking mode (extended reasoning) was disabled; enabling it may affect quantization sensitivity
- Mixture-of-Expert architectures may exhibit different quantization behavior
- Models larger than 5B parameters were not evaluated

6.5 Future Work

1. **Hardware power measurement**: Using external power meters or Apple's `powermetrics` to obtain ground-truth energy data on Apple Silicon.
2. **Broader benchmark coverage**: Extending evaluation beyond MMLU to reasoning, coding, long-context, and open-ended generation benchmarks to test whether the same quantization trends hold outside multiple-choice QA.
3. **Cross-architecture evaluation**: Extending experiments to NVIDIA GPUs, mobile NPUs, and embedded processors.
4. **Dynamic quantization**: Exploring per-layer or per-block adaptive bit-width selection based on sensitivity analysis.
5. **Combined techniques**: Evaluating quantization combined with structured pruning or knowledge distillation for further compression.

7 Conclusion

We set out to figure out how well post-training quantization works for deploying small language models on Apple Silicon. We ran the Qwen3.5 series (0.8B, 2B, 4B) and Phi-3 Mini through MLX at three precision levels (FP16, INT8, INT4) on an M4 Pro with 24 GB of unified memory.

Here is what we found:

1. **Throughput**: INT4 provides 2.0×–3.8× throughput improvement over FP16. The speedup ratio increases with model size, consistent with larger models being more memory-bandwidth-bound.
2. **Memory efficiency**: INT4 delivers 3.5×–3.6× memory reduction, with all models fitting within 2.5 GB, enabling deployment on devices with 8 GB RAM.
3. **Full-MMLU accuracy**: On all 57 subjects (14,042 questions), INT8 is effectively lossless for the 2B and larger models, while INT4 causes measurable drops of 2.39–7.90 pp depending on the model. The 4B-class models absorb INT4 best, losing 2.39–3.29 pp overall.

If there is one takeaway from this work, it is this: on Apple Silicon, INT4 quantization via MLX is the best speed–memory option when a modest accuracy trade-off is acceptable, especially for 4B-class models. When accuracy is the primary constraint, INT8 is the safer near-lossless default. The broader lesson is that aggressive quantization should be chosen per model, not per bit-width alone.

Code and Data: Available at https://github.com/harshpreet931/researchPaper-quantization

Acknowledgments

We acknowledge the developers of Hugging Face Transformers, CodeCarbon, and the Apple MLX team for making their tools openly available.

References

Marah Abdin et al. Phi-3 technical report: A highly capable language model locally on your phone. *arXiv preprint arXiv:2404.14219*, 2024.

Afsara Benazir and Felix Xiaozhu Lin. Profiling large language model inference on apple silicon: A quantization perspective. *arXiv preprint arXiv:2508.08531*, 2025.

Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33:1877–1901, 2020.

Yu Cheng, Duo Wang, Pan Zhou, and Tao Zhang. A survey of model compression and acceleration for deep neural networks. *arXiv preprint arXiv:1710.09282*, 2017.

Giuseppe Conti, David Jimenez, Alberto del Rio, Sandra Castano-Solis, Javier Serrano, and Jesus Fraile-Ardanuy. A multi-port hardware energy meter system for data centers and server farms monitoring. *Sensors*, 23(1):119, 2023. doi: 10.3390/s23010119.

Howard David, Eugene Gorbatov, Ulf R Hanebutte, Rahul Khanna, and Chi Le. Rapl: Memory power estimation and capping. In *Proceedings of the 2010 ACM/IEEE International Symposium on Low-Power Electronics and Design (ISLPED)*, pages 189–194, 2010.

Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. Llm.int8(): 8-bit matrix multiplication for transformers at scale. In *Advances in Neural Information Processing Systems*, volume 35, pages 30318–30332, 2022.

Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. Gptq: Accurate post-training quantization for generative pre-trained transformers. In *International Conference on Learning Representations*, 2023.

Leo Gao, Jonathan Tow, Baber Abbasi, Stella Biderman, Sid Black, Anthony DiPofi, Charles Foster, Laurence Golding, Jeffrey Hsu, Alain Le Noac'h, Haonan Li, Kyle McDonell, Niklas Muennighoff, Chris Ociepa, Jason Phang, Laria Reynolds, Hailey Schoelkopf, Aviya Skowron, Lintang Sutawika, Eric Tang, Anish Thite, Ben Wang, Kevin Wang, and Andy Zou. A framework for few-shot language model evaluation, 2024. URL https://github.com/EleutherAI/lm-evaluation-harness.

Georgi Gerganov. llama.cpp: Inference of llama models in pure c/c++. https://github.com/ggerganov/llama.cpp, 2023.

Song Han, Huizi Mao, and William J Dally. Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding. In *International Conference on Learning Representations*, 2016.

Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. In *International Conference on Learning Representations*, 2021.

Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. In *NIPS Deep Learning and Representation Learning Workshop*, 2015.

Donghao Huang and Zhaoxia Wang. Llms at the edge: Performance and efficiency evaluation with ollama on diverse hardware. In *2025 International Joint Conference on Neural Networks (IJCNN)*, 2025.

International Energy Agency. Energy and ai. https://www.iea.org/reports/energy-and-ai, 2025. Accessed: 2025-04-10.

Mojan Javaheripi and Sébastien Bubeck. Phi-2: The surprising power of small language models. Microsoft Research Blog, 2023.

Renren Jin, Jiangcun Du, Wuwei Huang, Wei Liu, Jian Luan, Bin Wang, and Deyi Xiong. A comprehensive evaluation of quantization strategies for large language models. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 12186–12215, 2024.

Eldar Kurtic, Alexandre Marques, Shubhra Pandit, Mark Kurtz, and Dan Alistarh. "give me bf16 or give me death"? accuracy-performance trade-offs in llm quantization. *arXiv preprint arXiv:2411.02355*, 2024.

Yann LeCun, John S Denker, and Sara A Solla. Optimal brain damage. *Advances in Neural Information Processing Systems*, 2:598–605, 1990.

Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. Awq: Activation-aware weight quantization for llm compression and acceleration. In *Proceedings of Machine Learning and Systems*, volume 6, pages 87–100, 2024. Best Paper Award.

ML CO2 Impact. Codecarbon: Track emissions from compute and recommend ways to reduce their impact on the environment. https://github.com/mlco2/codecarbon, 2023.

MLC Team. Mlc-llm: Universal deployment of large language models. https://github.com/mlc-ai/mlc-llm, 2023.

OpenAI. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.

David Patterson, Joseph Gonzalez, Quoc Le, Chen Liang, Lluis-Miquel Munguia, Daniel Rothchild, David So, Maud Texier, and Jeff Dean. Carbon emissions and large neural network training. *arXiv preprint arXiv:2104.10350*, 2021.

Weisong Shi, Jie Cao, Quan Zhang, Youhuizi Li, and Lanyu Xu. Edge computing: Vision and challenges. *IEEE Internet of Things Journal*, 3(5):637–646, 2016.

Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. Smoothquant: Accurate and efficient post-training quantization for large language models. In *International Conference on Machine Learning*, pages 38087–38099, 2023.

An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025. Qwen3.5 builds upon this architecture; no separate technical report available at time of writing.